

Rapport de projet

Prédiction automatique de paramètres de synthétiseur à partir de l'audio

Amaury Benard

Table des matières

Rapport de projet.....	1
Prédiction automatique de paramètres de synthétiseur à partir de l'audio.....	1
1. Introduction et contexte du projet.....	3
2. Vue d'ensemble de la chaîne de traitement.....	4
3. Organisation des fichiers et scripts.....	5
3.1 Scripts principaux.....	5
3.2 Fichiers de données.....	5
4. Génération et nettoyage des données.....	7
4.1 Génération des données.....	7
4.2 Nettoyage audio.....	7
4.3 Nettoyage des paramètres.....	7
5. Extraction des caractéristiques audio.....	8
6. Modèle de machine learning.....	9
6.1 Type de modèle.....	9
6.2 Justification du choix.....	9
6.3 Entraînement.....	9
7. Inférence.....	10
7.1 Inférence simple.....	10
7.2 Inférence multiple.....	10
8. Évaluation du modèle.....	11
8.1 Évaluation locale.....	11
8.2 Évaluation globale.....	11
9. Limites et perspectives.....	12
10. Conclusion.....	13

1. Introduction et contexte du projet

La création sonore par synthèse repose historiquement sur la manipulation manuelle d'un grand nombre de paramètres : oscillateurs, filtres, enveloppes, modulations, effets, etc. Ces paramètres sont généralement regroupés sous la forme de presets, qui constituent une représentation abstraite et numérique d'un son.

Dans la pratique, le lien entre un son et les paramètres qui l'ont généré est loin d'être intuitif. Deux presets très différents peuvent produire des sons perceptivement proches, tandis qu'une modification minimale d'un paramètre peut entraîner une variation sonore importante. Cette non-linéarité rend la conception sonore complexe, longue, et fortement dépendante de l'expérience de l'utilisateur.

L'objectif de ce projet est de s'attaquer à cette problématique en concevant une chaîne complète permettant d'estimer automatiquement des paramètres de synthétiseur à partir d'un signal audio. Plus précisément, à partir d'un son rendu par un preset, le système doit être capable de prédire les valeurs numériques des paramètres ayant permis de générer ce son.

Ce problème peut être vu comme une forme de synthèse inversée : au lieu de générer de l'audio à partir de paramètres (problème classique et bien maîtrisé), il s'agit ici de remonter de l'audio vers les paramètres de contrôle. Une telle approche présente plusieurs intérêts concrets :

- assistance à la création sonore, en proposant automatiquement un preset proche d'un son de référence,
- compréhension et analyse de presets existants à partir de leur rendu audio,
- accélération du sound design pour les musiciens et producteurs,
- ouverture vers des systèmes de contrôle audio-paramètres plus intuitifs.

D'un point de vue ingénierie, ce problème est particulièrement intéressant car il se situe à l'intersection de plusieurs domaines :

- traitement du signal audio,
- représentation temps-fréquence,
- apprentissage automatique supervisé,
- interaction homme-machine dans des environnements musicaux.

L'enjeu principal du projet n'est pas uniquement de proposer un modèle de machine learning performant, mais de concevoir une chaîne de traitement complète, cohérente et exploitable, depuis la génération des données jusqu'à l'évaluation des résultats.

Dans une première approche, l'idée initiale était de travailler directement au niveau du code interne de synthétiseurs logiciels existants tels que Vital ou d'autres VST comparables. Cette approche aurait permis un contrôle très fin du moteur de synthèse, mais elle s'est révélée rapidement peu

réaliste dans le cadre du projet, notamment en raison de la complexité des environnements C++/JUCE, du caractère propriétaire de nombreux synthétiseurs, et de la difficulté à automatiser proprement la génération de couples (audio, paramètres).

Le projet a donc été réorienté vers une solution plus pragmatique et plus adaptée aux contraintes : l'utilisation de Max for Live comme interface de génération de données. Ce choix a permis de conserver un contrôle précis sur les paramètres, tout en facilitant l'automatisation, la collecte des données et l'intégration avec les outils de traitement audio et de machine learning.

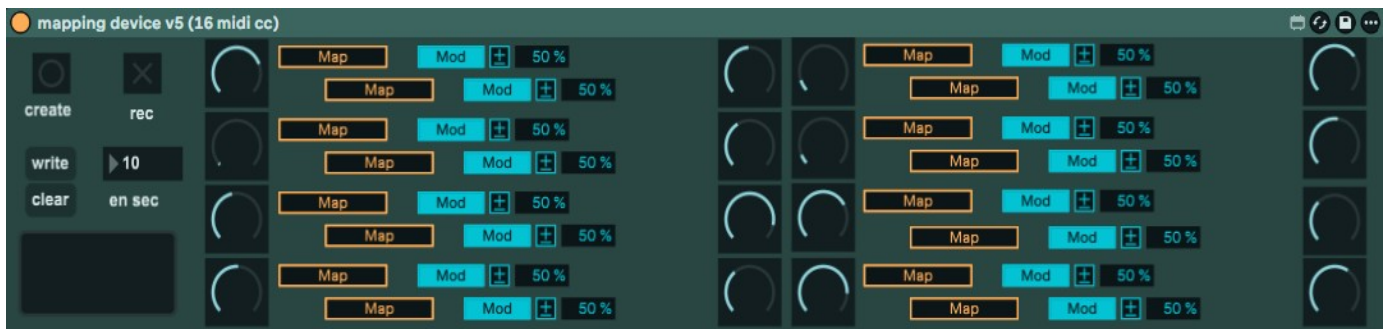
2. Outils utilisés et environnement de travail

Le projet repose sur l'intégration de plusieurs outils complémentaires, chacun répondant à un besoin précis de la chaîne de traitement. L'objectif n'est pas de développer un système isolé, mais de construire un environnement cohérent permettant de passer efficacement de la création sonore à l'apprentissage automatique.

2.1 Ableton Live et Max for Live

Ableton Live est utilisé comme environnement central de production audio. Il permet la gestion des pistes, l'automatisation et l'intégration native de Max for Live.

Max for Live est un environnement de programmation visuelle intégré à Ableton Live. Il permet de créer des devices personnalisés capables d'interagir directement avec les paramètres des instruments. Dans ce projet, Max for Live permet l'accès direct aux paramètres des synthétiseurs, l'automatisation de leur variation, la génération contrôlée de presets et l'export simultané de l'audio et des paramètres associés.



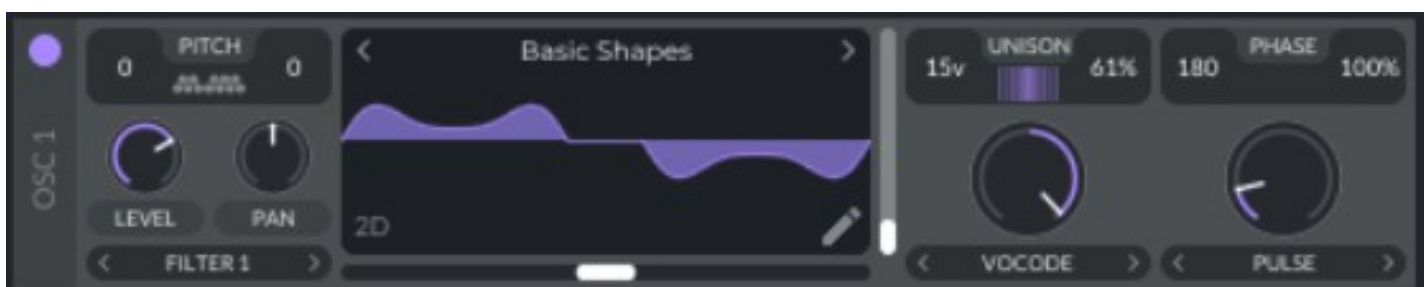
2.2 Synthétiseur Vital

Vital est un synthétiseur logiciel de type wavetable. Il dispose d'un grand nombre de paramètres continus et d'une architecture standard (oscillateurs, filtres, enveloppes, LFO). Il est particulièrement adapté au projet car ses paramètres sont bien définis, normalisables et fortement corrélés au rendu audio.



Paramètres visées :

Oscillateur,



Filtre,

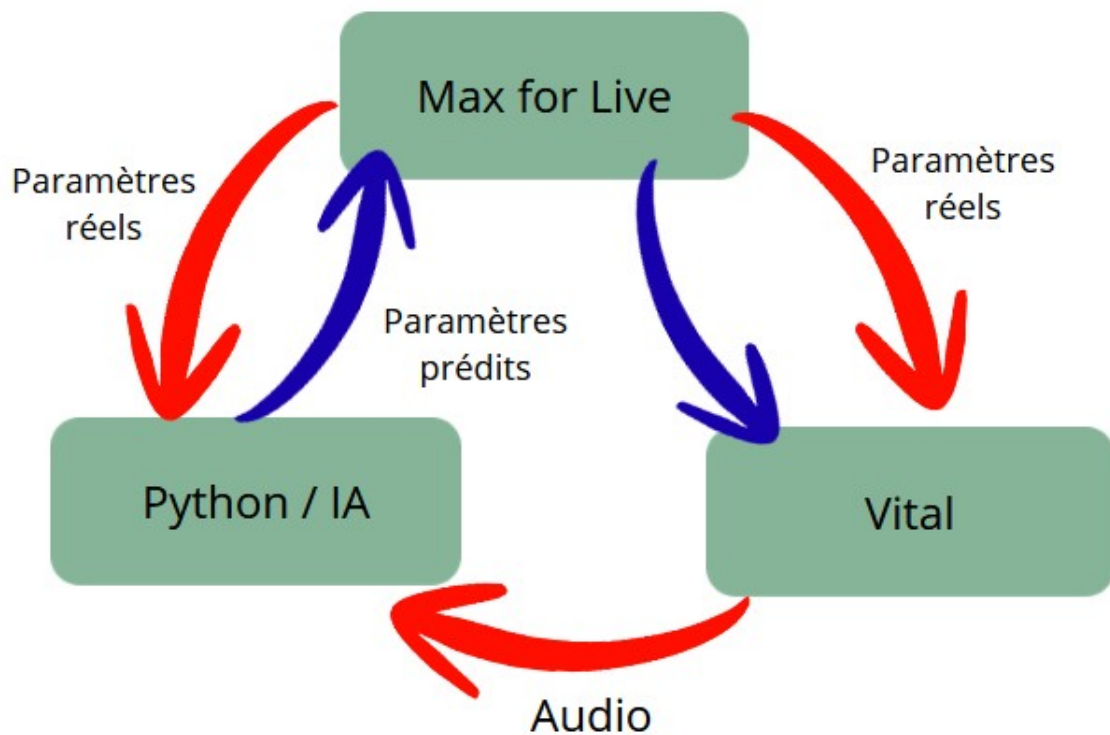
Enveloppe



2.3 Outils de traitement et d'apprentissage

La partie traitement du signal et apprentissage automatique est réalisée en Python, à l'aide de bibliothèques spécialisées : librosa pour l'audio, NumPy pour le calcul numérique et PyTorch pour la modélisation et l'entraînement du réseau de neurones.

On obtient le schéma global du flux de données suivant :



3. Organisation des fichiers et scripts

L'arborescence du projet reflète directement les différentes étapes du pipeline.

3.1 Scripts principaux

1_datacreator.py

Génération automatisée des données (liaison Max for Live / presets / audio).

1.2_preset_audio_cleaner.py

Nettoyage des fichiers audio trop silencieux ou inutilisables.

2_feature_extractor.py

Extraction des Mel-spectrogrammes à partir des audios.

2.2_optionnel_visualize_features.py

Visualisation des caractéristiques extraites (debug et validation).

3.2_uniformisator.py

Uniformisation des niveaux audio (volume) pour garantir une cohérence du dataset.

3.3_preset_txt_cleaner.py

Suppression des presets correspondant aux audios supprimés.

3.4_train.py

Entraînement du modèle de prédiction.

4_inference.py

Inférence sur un audio unique.

4_inference_multi.py

Inférence sur un long audio découpé en segments multiples.

5.2_evaluate_params.py

Évaluation simple sur un preset unique.

5.3_evaluate_params_multi.py

Évaluation globale sur plusieurs presets.

5_compare_audio_presets.py

Comparaison audio-audio entre preset réel et preset reconstruit.

3.2 Fichiers de données

features/ : Mel-spectrogrammes pré-calculés

presets/ : fichiers de presets utilisés pour la génération

params_fixed.txt : paramètres nettoyés et alignés

test_multi.txt : paramètres réels pour l'évaluation

predicted_params_multi.txt : paramètres prédits par le modèle

audio_to_params.pth : modèle entraîné

4. Génération et nettoyage des données

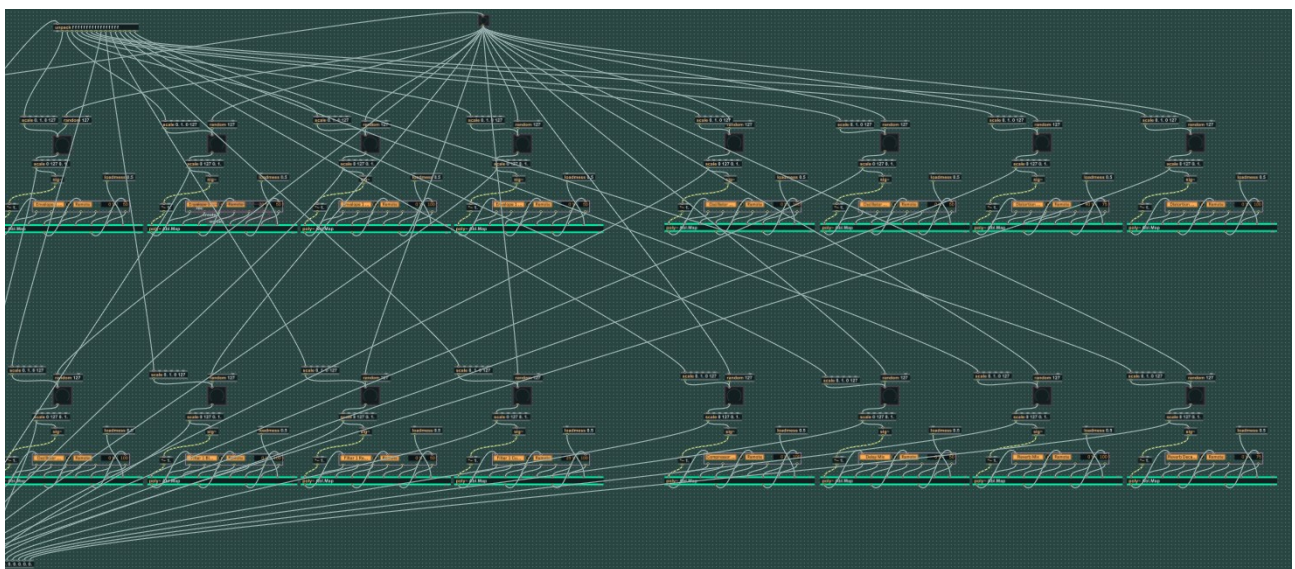
4.1 Génération des données via Max for Live

Les données utilisées pour l'entraînement du modèle sont générées automatiquement à l'aide d'un device Max for Live spécifiquement conçu pour ce projet. La création de ce device était nécessaire afin de maîtriser précisément le lien entre les paramètres du synthétiseur, les presets générés et les signaux audio associés, tout en automatisant la constitution du dataset.

Le device est structuré en trois blocs fonctionnels distincts, chacun ayant un rôle bien défini dans la chaîne de génération des données.



1) Bloc de mapping des paramètres



Ce premier bloc permet de définir et de contrôler les 16 paramètres du synthétiseur utilisés dans le projet.

Chaque paramètre est mappable dynamiquement vers un contrôle de Vital, ce qui garantit une compatibilité flexible avec différents presets ou configurations du synthétiseur.

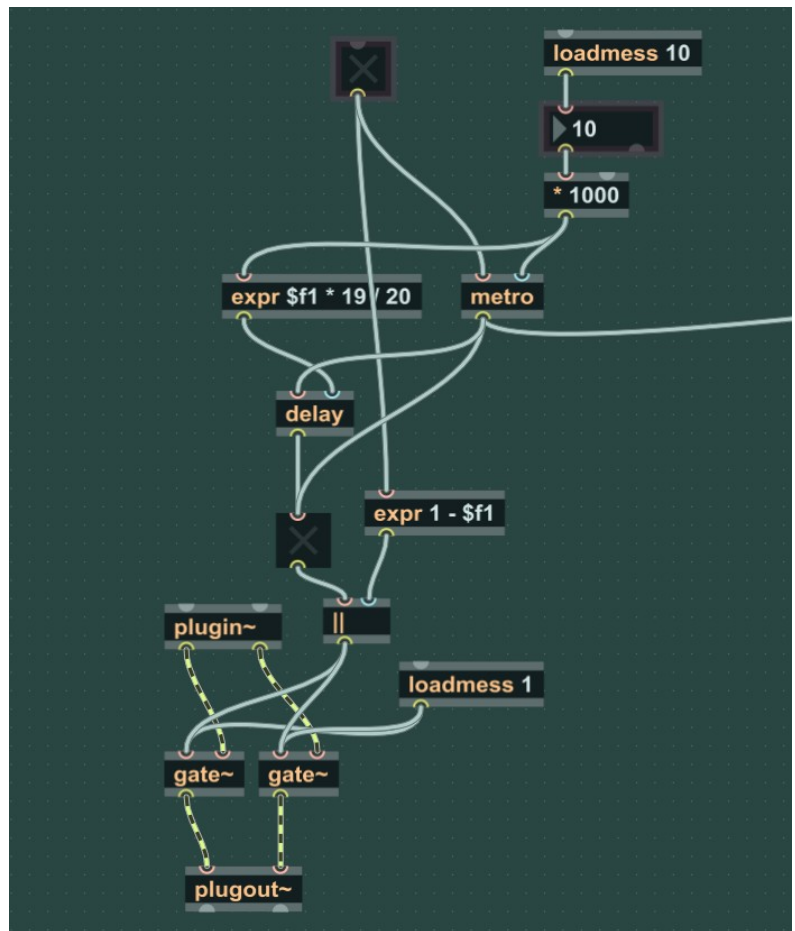
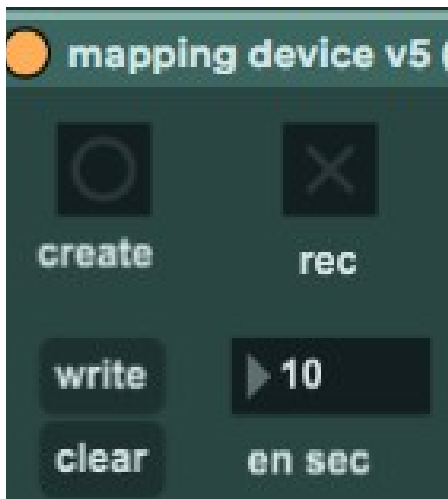


Le device génère pour ces 16 paramètres des valeurs aléatoires normalisées, comprises entre 0 et 1. À chaque génération de preset, les valeurs exactes des paramètres sont :

- affichées dans l'interface,
- enregistrées dans une mémoire interne,
- associées explicitement au rendu audio correspondant.

Cette étape est cruciale car elle assure une correspondance exacte entre le signal audio et les paramètres numériques, condition indispensable à l'apprentissage supervisé du modèle.

2) Bloc de génération des presets



Le second bloc est responsable de la création automatique des presets. Deux modes de fonctionnement sont proposés :

Mode manuel :

L'utilisateur peut générer un preset unique en appuyant sur un bouton nommé *Create*.

Cette action déclenche :

- la génération de 16 nouveaux paramètres aléatoires,
- l'application immédiate de ces paramètres au synthétiseur,
- la sauvegarde des valeurs générées dans la mémoire du device.

Mode automatique (Rec) :

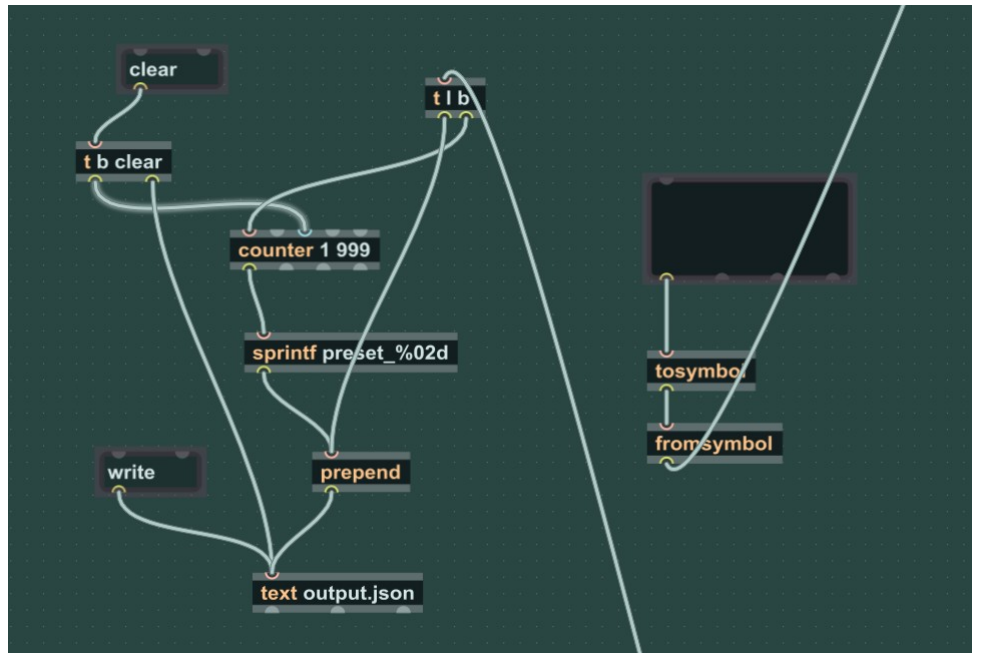
En activant le levier *Rec*, le device entre dans un mode de génération continue.

Dans ce mode, un nouveau preset est généré toutes les x secondes, où x est défini par une boîte numérique.

Afin de faciliter le découpage automatique des fichiers audio et de marquer clairement les transitions entre presets, le device coupe volontairement le son pendant $1/20^e$ de la durée totale du segment audio, juste avant chaque changement de preset.

Cette coupure permet de créer un silence identifiable servant de repère lors du traitement ultérieur des données audio.

3) Bloc d'enregistrement et de gestion des presets



Le troisième bloc gère la sauvegarde, l'édition et l'importation des presets générés :

- Le bouton *Write* permet d'exporter et d'enregistrer les presets récemment générés ainsi que leurs paramètres associés.
- Le bouton *Clear* efface la mémoire interne du device afin de repartir sur une nouvelle session de génération.
- Une boîte de texte éditable permet d'écrire ou de coller directement un preset sous forme textuelle.

Cette fonctionnalité est utilisée pour importer des presets existants, notamment ceux issus des paramètres prédits par le modèle, afin de générer les fichiers audio correspondants et d'évaluer la qualité des prédictions.

Chaque preset généré correspond à un segment audio de durée fixe (par exemple 5 secondes), enregistré simultanément avec ses 16 paramètres numériques.

Cette approche garantit un dataset parfaitement structuré, cohérent et exploitable pour l'entraînement et l'évaluation du modèle de prédiction.

📁 preset_002	
📁 preset_003	
📁 preset_004	
📁 preset_005	
📁 preset_006	
📁 preset_007	
📁 preset_008	

preset_01	0.433	0.283	0.150	0.276
preset_05	0.346	0.630	0.661	0.488
preset_09	0.024	0.646	0.756	0.717
preset_13	0.370	0.543	0.551	0.732
preset_17	0.504	0.472	0.646	0.472
preset_21	0.803	0.953	0.772	0.795
preset_25	0.283	0.346	0.000	0.016
preset_29	0.386	0.992	0.803	0.417
preset_33	0.134	0.803	0.071	0.835
preset_37	0.362	0.118	0.724	0.724
preset_41	0.244	0.646	0.543	0.024
preset_45	0.465	0.441	0.236	0.882

4.2 Nettoyage audio

Le script **preset_audio_cleaner.py** supprime automatiquement les fichiers audio dont le niveau RMS est trop faible. Ces fichiers sont considérés comme inutilisables car ils faussent l'apprentissage.

Un fichier de log est généré pour tracer précisément quels audios ont été supprimés.

4.3 Nettoyage des paramètres

Le script **preset_txt_cleaner.py** garantit la cohérence entre les audios conservés et les paramètres associés

Tout preset dont l'audio a été supprimé est automatiquement retiré du fichier de paramètres.

5. Extraction des caractéristiques audio

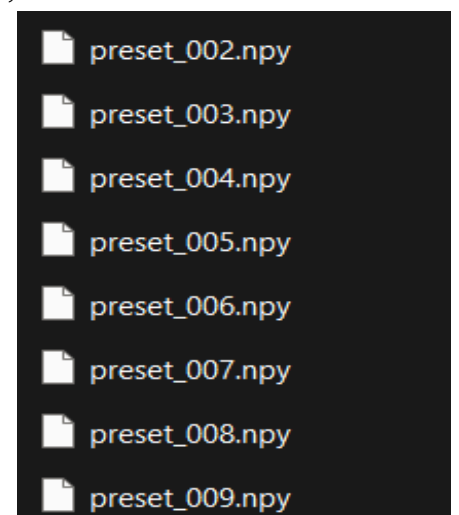
Les audios nettoyés sont transformés en Mel-spectrogrammes via librosa.

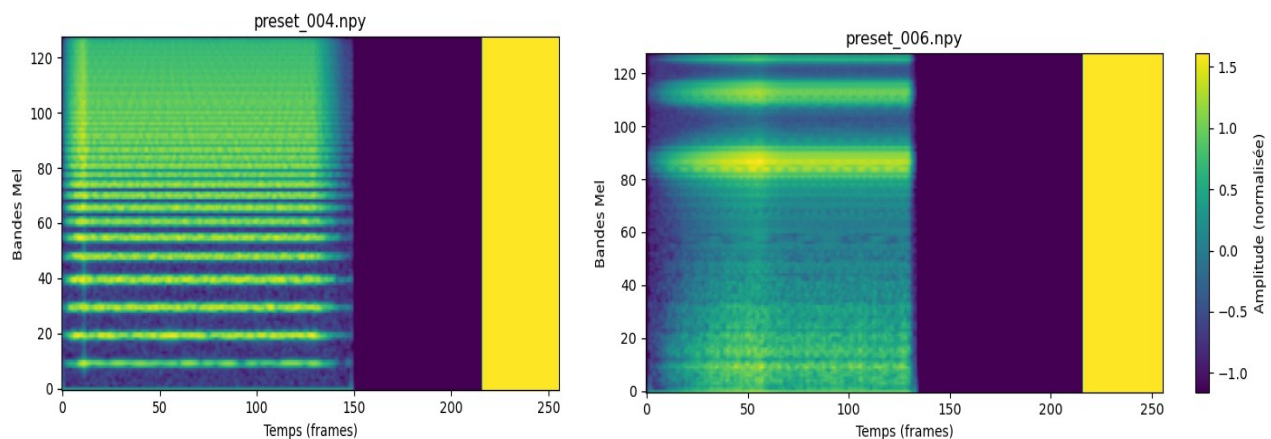
Choix techniques :

- représentation temps-fréquence adaptée à la perception humaine,
- taille fixe (padding ou découpe),
- normalisation statistique.

Les Mel-spectrogrammes sont sauvegardés dans le dossier

features/ afin d'éviter de recalculer ces données à chaque entraînement.





6. Modèle de machine learning

6.1 Type de modèle

Le modèle utilisé est un réseau de neurones convolutif (CNN) pour la régression.

- Entrée : Mel-spectrogramme (image 2D)
- Sortie : vecteur de paramètres normalisés

6.2 Justification du choix

- Les CNN sont particulièrement adaptés aux données audio représentées sous forme spectrale.
- Ils permettent de capter des motifs locaux (harmoniques, transitoires).
- Le modèle reste léger, stable et interprétable.

6.3 Entraînement

L'entraînement est effectué sur 50 époque avec :

- fonction de perte : MSE
- optimiseur : Adam
- apprentissage supervisé

Le modèle final est sauvegardé sous forme de fichier .pth.

Epoch 1		Loss: 0.078331
Epoch 2		Loss: 0.064957
Epoch 3		Loss: 0.059740
Epoch 4		Loss: 0.055072
Epoch 5		Loss: 0.051712
Epoch 6		Loss: 0.048521
Epoch 7		Loss: 0.045277
Epoch 8		Loss: 0.040493
Epoch 9		Loss: 0.037001
Epoch 10		Loss: 0.032702

 audio_to_params.pth

7. Inférence

7.1 Inférence simple

Le script `inference.py` permet de prédire les paramètres à partir d'un seul fichier audio.

7.2 Inférence multiple

Le script `inference_multi.py` prend un audio long, le découpe en segments fixes, et génère une suite de presets prédits, compatibles avec les fichiers de paramètres du projet.

8. Évaluation du modèle

L'évaluation du modèle a pour objectif de mesurer sa capacité à prédire des paramètres de synthétiseur cohérents à partir d'un signal audio, mais aussi d'analyser ses limites comportementales. Deux niveaux d'évaluation ont été mis en place : une évaluation locale, preset par preset, et une évaluation globale sur l'ensemble du jeu de test.

8.1 Évaluation locale

L'évaluation locale repose sur une comparaison directe, pour chaque preset, entre les paramètres réels utilisés pour générer le son et les paramètres prédits par le modèle à partir de l'audio correspondant.

Cette comparaison est réalisée à l'aide de scripts dédiés permettant de charger simultanément les paramètres de référence et les paramètres prédits, puis de calculer différentes métriques d'erreur.

Les métriques utilisées sont les suivantes :

- **MAE (Mean Absolute Error)** : moyenne des erreurs absolues sur l'ensemble des paramètres. Cette métrique donne une idée intuitive de l'écart moyen entre les valeurs prédites et les valeurs réelles.
- **RMSE (Root Mean Squared Error)** : pénalise davantage les erreurs importantes et permet de détecter des écarts significatifs sur certains paramètres.
- **Score de similarité** : métrique complémentaire permettant d'estimer la proximité globale entre deux vecteurs de paramètres, indépendamment des erreurs locales isolées.

```
preset_002
MAE : 0.214690
RMSE : 0.284963
Score: 78.53 / 100
Interprétation : Proche
```

```
preset_003
MAE : 0.200889
RMSE : 0.258885
Score: 79.91 / 100
Interprétation : Proche
```

```
preset_005
MAE : 0.240067
RMSE : 0.332401
Score: 75.99 / 100
Interprétation : Proche
```

```
preset_006
MAE : 0.209462
RMSE : 0.257164
Score: 79.05 / 100
Interprétation : Proche
```

Cette évaluation locale permet d'identifier les paramètres les mieux estimés par le modèle, mais aussi ceux qui posent davantage de difficultés, notamment lorsque leur impact perceptif sur le son est plus subtil.

8.2 Évaluation globale

Pour évaluer les performances générales du modèle et sa capacité de généralisation, une évaluation globale a été réalisée sur un jeu de test indépendant, créé spécifiquement pour cette étape et non utilisé lors de l'entraînement.

- **Nombre de presets évalués : 120**
- **MAE moyen : 0.206**
- **RMSE moyen : 0.267**

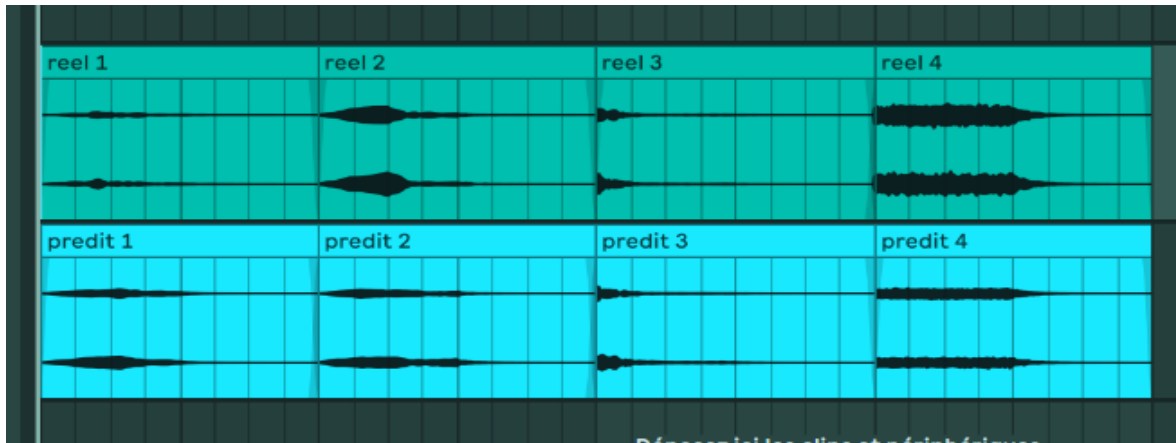
Ces valeurs indiquent que, en moyenne, l'erreur sur chaque paramètre reste relativement modérée, compte tenu du fait que les paramètres sont normalisés dans l'intervalle $[0,1]$. Un MAE d'environ 0.2 signifie que le modèle se trompe en moyenne d'environ 20 % de la plage totale d'un paramètre, ce qui reste acceptable pour une première approche sur un problème aussi ambigu que l'inversion audio → paramètres.

Cependant, une analyse qualitative des résultats met en évidence un comportement important du modèle :

celui-ci a tendance à prédire des paramètres "passe-partout", c'est-à-dire des valeurs moyennes produisant un son relativement neutre et stable. Cette stratégie permet de minimiser l'erreur globale et d'obtenir de bons scores numériques, mais se fait au détriment de la prise de risque sur des paramètres plus extrêmes ou plus caractéristiques.

Autrement dit, le modèle privilégie une solution robuste mais conservatrice, ce qui est cohérent avec l'utilisation d'une fonction de perte de type MSE et avec la nature fondamentalement non bijective du problème : plusieurs ensembles de paramètres peuvent produire des sons perceptuellement proches.

```
Nombre de presets comparés : 120
MAE moyen   : 0.205842
RMSE moyen  : 0.267472
Score moyen : 79.42 / 100
Interprétation globale : Proche
```

9. Limites et perspectives

Limites identifiées :

- ambiguïté audio → paramètres,
- dépendance à la qualité du dataset,
- absence de modélisation temporelle longue.

Perspectives possibles :

- modèles plus profonds ou hybrides (CNN + attention),
- pondération des paramètres,
- apprentissage perceptif audio–audio,
- intégration temps réel.

10. Conclusion

Ce projet avait pour objectif de concevoir une chaîne complète permettant d'estimer automatiquement des paramètres de synthétiseur à partir d'un signal audio. Les résultats obtenus montrent que cette approche est non seulement faisable, mais également prometteuse.

La chaîne développée couvre l'ensemble du processus : génération automatisée de données via Max for Live, extraction de caractéristiques audio, apprentissage d'un modèle convolutif, puis évaluation quantitative et qualitative des paramètres prédits. Le modèle atteint des performances cohérentes sur un jeu de test indépendant, avec des erreurs maîtrisées et une bonne stabilité globale.

Les limites observées, notamment la tendance du modèle à produire des paramètres moyens, ne constituent pas un échec mais plutôt une indication claire des pistes d'amélioration possibles : intégration de métriques perceptuelles, fonctions de coût orientées audio, modèles plus expressifs ou encore apprentissage multi-objectifs.

En pratique, l'outil développé possède un réel potentiel applicatif. Il pourrait servir de base à des systèmes d'assistance à la création sonore, de récupération de presets à partir d'exemples audio, ou encore d'outils pédagogiques pour comprendre l'influence des paramètres d'un synthétiseur. Le projet répond ainsi de manière pertinente à la problématique initiale, tout en ouvrant la voie à de nombreuses extensions.

Remerciements

Je tiens à remercier **Sylvain Reynal**, professeur encadrant de ce projet, pour son accompagnement tout au long du travail, ses conseils techniques et méthodologiques, ainsi que pour la liberté laissée dans l'exploration des différentes approches. Son encadrement a été déterminant dans la structuration du projet et dans la prise de recul sur les choix techniques effectués.